



Artefactos Scrum — Sprint 10

Tienda interna v1 — Abrir el catálogo de la tienda del edificio: el admin gestiona productos (alta/baja/stock/precio + foto) desde /admin/tienda y el residente navega /residente/tienda en grilla con filtros y búsqueda · Chore: CD a staging (deploy automático en merge a main) que cierra la verificación post-deploy de DoD v2

Producto	Zity
Sprint	Sprint 10 — Semana 12
Stack	React 19 + Vite 8 + TailwindCSS 4 · Supabase (Postgres + Auth + Storage + Realtime) · Resend · Vercel · GitHub Actions · Vitest · Recharts (desde S7) · pdf-lib (S9) · pg_cron (S9)
Product Owner	Alvarez Rocca Jaqueline
Scrum Master	Meza Pelaez Carlos
Developers	Cortez Zamora Leonardo Fabian · Gonza Morales Yoel Ronaldo · Santiago Flores Carlos Steven
Capacidad semanal	3 h/día × 5 integrantes = 15 horas/semana · 60 horas/mes
Horas estimadas	13 horas (2 horas de buffer)
DoD aplicable	DoD v2 — séptima aplicación (el chore CD cierra la verificación post-deploy automática)
Nuevo en este sprint	Módulo Tienda v1: tablas productos / pedidos / pedido_items + RLS por rol + bucket productos-fotos · vista /admin/tienda con CRUD del catálogo (alta/baja/stock/precio + foto a Storage) · vista /residente/tienda en grilla de tarjetas con filtros (categoría, disponibilidad) y búsqueda por nombre · indicador de stock bajo/agotado · CD: deploy automático a staging en cada merge a main (Vercel)

Chore técnico del Sprint	CD a staging con GitHub Actions (deploy automático en merge a main, con CI como gate y verificación post-deploy GET /health) que cierra el último criterio de DoD v2. Sin smoke tests más completos por ahora (entran como chore del S14). 2 h, P2.
Variables nuevas	VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID (Secrets de GitHub) para el deploy automático a Vercel
Mejoras del profesor	Addendum al Sprint 10 (≈8 h adicionales, recomendadas por el profesor e incorporadas al Sprint Planning, sin alterar el compromiso base de 13 h de la Tienda v1): MP-01 seed histórico de 3 años de datos de demostración · MP-02 pago en línea (simulado) de facturas por el residente (HU-FACT-09) · MP-03 encuesta de usabilidad (escala Likert tipo SUS) vía Google Form. Ver el Sprint Planning (§1).
Nota	Documento académico — Datos ficticios sin PII real

Documento vivo — se actualiza en cada Sprint Review

1 ACTA DE SPRINT PLANNING

Sprint	Sprint 10 — Semana 12
Fecha	Lunes — inicio de semana 12
Duración del evento	75 minutos + 30 min de Planning ampliado con stakeholder (Acción 2 del Retro S8, segunda aplicación)
Facilitador	Meza Pelaez Carlos (Scrum Master)
Asistentes	Alvarez Rocca Jaqueline (PO), Meza Pelaez Carlos (SM), Cortez Zamora Leonardo, Gonza Morales Yoel, Santiago Flores Carlos (Devs)
Stakeholder invitado	Laura Vega (Residente ficticia) — como futura compradora de la tienda, valida la navegación del catálogo (grilla, filtros, búsqueda) y qué información necesita ver de cada producto. Segunda aplicación de la Acción 2 del Retro S8.
Capacidad	15 horas disponibles · se usarán 13 h (2 h de buffer)
Entrada	Product Backlog tras Sprint 9 Review. Facturación v2 cerró el dominio de finanzas y /health cerró DoD v2. Entra el Epic TIENDA-01 v1 (catálogo). El chore de CD cierra la verificación post-deploy automática que quedaba pendiente de DoD v2.
Refinement previo	Sexta aplicación de la práctica. 'Edge cases del dominio' (Acción 3 del Retro S9): «¿qué pasa si el stock llega a 0?», «¿se puede dar de baja un producto con pedidos asociados?», «¿qué formato y peso de foto?». Incluye un spike de 15 min de la subida de fotos a Storage (Acción 2 del Retro S9).

"Sprint Goal: Abrir la tienda interna del edificio: el admin da de alta productos (con foto, stock y precio) y los gestiona desde /admin/tienda, y el residente navega el catálogo en /residente/tienda en una grilla de tarjetas con filtros por categoría y disponibilidad y búsqueda por nombre. En paralelo, el chore de CD deja el deploy a staging automático en cada merge a main."

Nota: Séptima aplicación de DoD v2. La Tienda abre el último dominio nuevo del producto. En este Sprint solo se entrega el catálogo (v1): el carrito, el descuento de stock y la integración con la factura mensual son el S11 (Tienda v2, primera aplicación de DoD v3). Las tablas pedidos y pedido_items se crean ahora con su RLS pero aún no tienen flujo de UI. El chore de CD cierra la verificación post-deploy automática que quedaba de DoD v2.

PBIs seleccionados — Sprint 10

ID	Historia / Tarea	Tipo	Prior.	Horas	Responsable
HU-TIENDA-01	Modelado BD: tablas productos, pedidos, pedido_items + RLS por rol + bucket productos-fotos	Historia	● P1	2 h	Gonza Morales

ID	Historia / Tarea	Tipo	Prior.	Horas	Responsable
HU-TIENDA-02	Vista /admin/tienda: CRUD del catálogo (alta/baja/stock/precio + foto a Storage)	Historia	● P1	3 h	Gonza Morales
HU-TIENDA-05	Vista /residente/tienda: catálogo en grilla de tarjetas (foto, nombre, precio, stock)	Historia	● P1	3 h	Santiago Flores
HU-TIENDA-06	Filtros del catálogo (categoría, disponibilidad) + búsqueda por nombre	Historia	● P2	2 h	Santiago Flores
Mejora	Indicador visual de stock bajo (≤ 5) y agotado (= 0) en la tarjeta del producto	Mejora	● P3	1 h	Santiago Flores
Chore-T	CD: deploy automático a staging en merge a main (GitHub Actions + Vercel) + verificación /health	Chore	● P2	2 h	Cortez Zamora
Buffer	Reservado para imprevistos del CRUD de fotos (Storage) o del workflow de CD	—	● —	2 h	—

Total estimado: 13 horas comprometidas + 2 h de buffer (techo de 15 h).

Mejoras recomendadas por el profesor — addendum al Sprint 10

El Profesor del curso (stakeholder académico) recomendó tres mejoras para acercar el producto a un escenario real y validar su usabilidad. Se incorporan al alcance del Sprint 10 ADEMÁS del compromiso base de 13 h de la Tienda v1 (≈8 h adicionales) y se colocan aquí, junto a los PBIs del Sprint, para verlas en conjunto. El núcleo del Sprint y su velocidad se mantienen; estas mejoras se reportan por separado.

"Objetivo del addendum: Addendum del Sprint 10 — (1) cargar 3 años de datos de demostración para dar realismo a las métricas y al dashboard ejecutivo; (2) permitir al residente pagar sus facturas en línea (pago simulado, listo para una pasarela real); (3) medir la usabilidad del sistema con una encuesta tipo SUS (Google Form)."

Resumen de las mejoras del profesor

ID	Mejora	Tipo	Prior.	Horas	Responsable
MP-01	Seed histórico: 3 años de datos de demostración (facturas, pagos, mantenimiento)	Chore datos	● P2	3 h	Cortez Zamora
MP-02	Pago en línea (simulado) de facturas por el residente — HU-FACT-09	Historia	● P1	4 h	Gonza / Santiago
MP-03	Encuesta de usabilidad (escala Likert tipo SUS) vía Google Form	Validación	● P2	1 h	Alvarez (PO)

Total addendum: 8 horas adicionales sobre las 13 h del núcleo del Sprint (Tienda v1). El desglose se ve en la tabla de impacto al final de este bloque.

MP-01 · Seed histórico — 3 años de datos de demostración

Qué resuelve: hoy el sistema solo tiene unos pocos registros del mes en curso (scripts/seed.js + seed-tiempo.js), por lo que el Dashboard de métricas (S7) y el futuro Dashboard ejecutivo del dueño (S14) se ven casi vacíos. El profesor pidió datos de varios años para que las tendencias y la demo final sean creíbles.

Cómo: un nuevo script idempotente **npm run seed:historico** (scripts/seed-historico.js) genera ~36 meses de datos ficticios para los residentes demo (correos @zity-demo.com):

- Facturas mensuales por residente (luz, agua, pensión) con montos en soles (S/) y una distribución realista de estados: la mayoría 'pagada' (con fecha_pago y método), algunas 'vencida' y otras 'pendiente'.
- Solicitudes de mantenimiento repartidas a lo largo de los 36 meses, con categorías, estados (resuelta / en proceso / pendiente) y técnico asignado, para poblar las métricas y gráficas del S7.
- Pagos coherentes con las facturas pagadas (vía registrar_pago_factura), para alimentar los totales del periodo (cobrado / pendiente / vencido) y la tendencia mensual de ingresos.

Decisiones técnicas — MP-01

Decisión	Detalle
Solo entorno demo/staging	El seed histórico nunca corre contra producción; usa la SUPABASE_SERVICE_ROLE_KEY del entorno demo y solo toca usuarios @zity-demo.com.
Idempotente y determinista	Re-ejecutar npm run seed:historico reinicia y regenera el histórico al mismo resultado (sin acumular duplicados), siguiendo el patrón ya usado en seed-tiempo.js.
Respetar RLS y constraints	Inserta vía service role respetando los CHECK de monto/estado/método y las claves foráneas; no desactiva RLS. Volumen acotado (~36 meses × residentes demo) para no degradar el dashboard.
Reusa el dominio existente	No crea tablas nuevas: usa facturas, solicitudes y audit_log ya existentes (S3–S9); solo añade el generador de datos.

MP-02 · Pago en línea de facturas por el residente (simulado)

Qué resuelve: hoy el único camino a 'pagada' es que el **admin** ejecute registrar_pago_factura (efectivo / transferencia / otro); el residente no puede pagar desde la app. El profesor sugirió una pasarela de pagos o alguna forma de que el residente pague sus facturas.

Enfoque (decisión del equipo): en esta iteración el pago es **simulado** (mock): un flujo de pago ficticio, sin dinero ni proveedor real, que marca la factura como pagada. Queda **listo para conectar una pasarela real** (Culqi, Izipay o MercadoPago Perú, ya que el sistema opera en soles) mediante una Edge Function + webhook de confirmación en un Sprint futuro. Así se evita manejar credenciales y datos de tarjeta reales en un proyecto académico.

Como **residente**, quiero **pagar mis facturas pendientes o vencidas desde la app mediante un pago en línea**, para no depender de que el administrador registre mi pago manualmente.

Criterios de aceptación

- En /residente/facturas, cada factura 'pendiente' o 'vencida' muestra un botón **Pagar**; las 'pagadas' no lo muestran.
- El botón abre un modal de pago **simulado**: muestra el monto en soles (S/) y un formulario de tarjeta ficticia (número / vencimiento / CVV) que NO se procesa contra ningún proveedor; un aviso indica que es un pago de demostración.
- Al confirmar, una RPC marca la factura como 'pagada' con metodo_pago='tarjeta', fecha_pago = hoy (America/Lima) y registrado_por = el propio residente; es idempotente (si ya está pagada, no reescribe).
- Seguridad: el residente solo puede pagar **sus propias** facturas (verificación de residente_id en la RPC + RLS); nunca las de otro residente.
- Reusa el trigger after_factura_paid (S9): se emite la notificación 'factura_pagada' y la factura se ve como pagada al instante (Realtime).
- El admin sigue viendo el pago en su panel y en los totales del periodo (cobrado), igual que un pago registrado manualmente.

***Evidencia:** En la Review, Laura pagó su factura de agua desde /residente/facturas con el pago simulado; la factura pasó a 'pagada', llegó la notificación y el monto se sumó a 'cobrado' en el panel del admin.*

***Nota técnica:** se amplía el CHECK de facturas.metodo_pago para admitir 'tarjeta' (además de efectivo/transferencia/otro) y se añade una RPC pagar_factura_residente (SECURITY DEFINER) que valida auth.uid() = residente_id. Migrar a una pasarela real solo requiere reemplazar la confirmación simulada por el webhook del proveedor; el resto del flujo (estado, notificación, totales) no cambia.*

MP-03 · Encuesta de usabilidad (Google Form, escala Likert tipo SUS)

Qué resuelve: hasta ahora la validación se apoyó en stakeholders ficticios (Laura, Carlos, Rosa). El profesor pidió medir con usuarios reales qué tan intuitivo y útil es el sistema, con un Google Form que incluya una breve descripción del sistema, el enlace y preguntas en escala Likert.

Instrumento: se adopta una encuesta tipo **SUS (System Usability Scale)** — estándar de la industria para medir usabilidad percibida — con 10 ítems en escala Likert de 1 (Totalmente en desacuerdo) a 5 (Totalmente de acuerdo), más dos preguntas abiertas. El SUS produce un puntaje de 0 a 100; un valor ≥ 68 se considera por encima del promedio.

- **Encabezado:** breve descripción de Zity (plataforma de gestión del edificio: mantenimiento, facturación, comunicación y tienda interna) + enlace al sistema en staging.
- **Datos del encuestado:** rol con el que probó (residente / administrador / técnico / visitante), sin datos personales sensibles (alineado con 'Datos ficticios sin PII real').
- **10 ítems Likert (1–5):** intuitividad, facilidad de uso, consistencia, confianza, rapidez para completar tareas y utilidad percibida de los módulos.
- **2 preguntas abiertas:** qué te resultó más útil y qué mejorarías.
- **Aplicación:** se difunde a compañeros y usuarios reales; se recogen las respuestas y el puntaje SUS se reporta en el S13/S14 como evidencia de validación.

El contenido completo del formulario (descripción, ítems y escala, listo para copiar a Google Forms) está en el Anexo A de este bloque (más abajo) y, además, en el archivo docs/sprints/Zity_Encuesta_Usabilidad_GoogleForm.md del repositorio.

Anexo A · Contenido del Google Form de usabilidad

Título: Encuesta de usabilidad — Zity (gestión de tu edificio)

Descripción (encabezado del formulario): Zity es una plataforma web para gestionar tu edificio: reportar y seguir solicitudes de mantenimiento, recibir y pagar tus facturas (luz, agua, pensión), comunicarte con la administración y comprar en la tienda interna. Pruébalo en el enlace y cuéntanos qué tan fácil e intuitivo te resultó. Te tomará unos 3 minutos. Tus respuestas son anónimas y con fines académicos.

Enlace al sistema: <https://zity.vercel.app> (reemplazar por la URL real de staging)

Sección 1 — Sobre ti: ¿Con qué rol probaste Zity? (opción única: Residente / Administrador / Técnico / Solo exploré)

Sección 2 — Escala de usabilidad (1 = Totalmente en desacuerdo ... 5 = Totalmente de acuerdo):

1. Creo que me gustaría usar Zity con frecuencia.
2. Encontré el sistema innecesariamente complejo.
3. Me pareció fácil de usar.
4. Creo que necesitaría ayuda de una persona técnica para poder usar Zity.
5. Las funciones del sistema (mantenimiento, facturas, tienda) están bien integradas.
6. Encontré demasiada inconsistencia en el sistema.
7. Imagino que la mayoría de la gente aprendería a usar Zity muy rápido.
8. Me pareció muy engorroso de usar.
9. Me sentí seguro/a al usar el sistema.
10. Necesité aprender muchas cosas antes de poder manejar Zity.

Sección 3 — Preguntas abiertas:

- ¿Qué fue lo que te resultó más útil o lo que más te gustó?
- ¿Qué cambiarías o mejorarías para que el sistema sea más fácil de usar?

Cálculo del puntaje SUS: en los ítems impares (1, 3, 5, 7, 9) reste 1 a la respuesta; en los pares (2, 4, 6, 8, 10) reste la respuesta a 5. Sume los 10 valores (rango 0–40) y multiplique por 2.5 para obtener un puntaje de 0 a 100. Referencia de la industria: 68 = promedio.

Impacto en la capacidad del Sprint 10

Concepto	Horas
Núcleo del Sprint 10 — Tienda interna v1 + CD (sin cambios)	13 h
MP-01 · Seed histórico de 3 años	3 h
MP-02 · Pago en línea simulado de facturas (HU-FACT-09)	4 h
MP-03 · Encuesta de usabilidad (Google Form)	1 h
Total Sprint 10 con addendum	21 h

El núcleo (13 h) conserva su estimación, su velocidad y la racha 'sin hotfixes'. El addendum (8 h) se reconoce como esfuerzo adicional puntual aceptado a pedido del profesor; no se promedia con la velocidad histórica para no distorsionarla.

Decisiones técnicas del Sprint

Antes de empezar el desarrollo, el equipo cierra las siguientes decisiones técnicas:

Decisión	Detalle	Registrado en
Modelo de la tabla productos	Columnas: id (uuid), nombre (text), descripcion (text), categoria (enum: bebidas/comestibles/limpieza/otros), precio (numeric(10,2) ≥ 0), stock (integer ≥ 0), activo (boolean, default true), imagen_url (text → bucket productos-fotos), created_at, updated_at. La baja es lógica (activo=false), no DELETE.	Migración 011 + ADR-011
Tablas pedidos y pedido_items	Se crean ahora (con RLS) y se usan en el S11. pedidos: id, residente_id (FK profiles), estado (enum: borrador/confirmado/facturado), total (numeric(10,2)), periodo (text), created_at. pedido_items: id, pedido_id (FK), producto_id (FK), cantidad (int ≥ 1), precio_unitario (numeric(10,2), snapshot al pedir). Sin UI de carrito hasta el S11. El PRD §4.4 las proyectaba para el S11; se modelan ya en el S10 (solo BD + RLS, sin UI) para evitar una migración posterior.	Migración 011 + ADR-011
RLS de la Tienda	productos: SELECT para cualquier usuario autenticado donde activo=true (residente y técnico ven el catálogo); admin SELECT/INSERT/UPDATE en todo. pedidos/pedido_items: residente solo los suyos, admin todos, técnico sin acceso. Cada política se cubre con un test de los 3 roles.	Migración 011 (RLS)
Foto del producto	Bucket productos-fotos en Supabase Storage (reusa el patrón de solicitudes-fotos del S3). Subida desde /admin/tienda con validación (JPEG/PNG, ≤ 2 MB) y URL firmada para mostrar. Sin foto → placeholder.	ADR-011 + /docs/storage.md
Baja de producto y stock 0	Baja = activo=false (no DELETE): desaparece del catálogo del residente pero queda para el historial de pedidos del S11; la FK pedido_items → productos usa ON DELETE RESTRICT. Stock 0 = visible con badge 'Agotado' y deshabilitado para compra (la validación de compra es del S11).	Criterio HU-TIENDA-02 + conventions.md
CD a staging	Workflow deploy-staging.yml (GitHub Actions): en cada merge a main, build + deploy a Vercel con VERCEL_TOKEN/ORG_ID/PROJECT_ID. El job de CD depende del de CI (needs:) y solo despliega si lint + tests pasan; verificación post-deploy con GET /health. Cierra DoD v2. Sin smoke tests aún (chore del S14).	Chore-T + ADR-012 + /docs/ops/cd-staging.md

Desglose de tareas — ¿Cómo?

Gonza Morales Yoel — HU-TIENDA-01 + HU-TIENDA-02 (5 h)

Tarea	Horas
Migración 011: tablas productos, pedidos, pedido_items + enums (categoria, estado de pedido) + constraints (precio ≥ 0, stock ≥ 0, cantidad ≥ 1, FK con ON DELETE RESTRICT) + índices por (categoria, activo) y (residente_id).	1 h
Políticas RLS de la Tienda (productos: lectura autenticada de activos, admin todo; pedidos/items por residente) + bucket productos-fotos con políticas + tests de integración con los 3 roles.	1 h

Tarea	Horas
Vista /admin/tienda: listado de productos (activos e inactivos) + formulario de alta/edición (nombre, categoría, precio, stock). Validaciones cliente + servidor. Registro en audit_log.	2 h
Subida de foto a productos-fotos (preview + validación de tipo/peso + URL firmada) + baja lógica (activo=false) con opción de reactivar.	1 h

Santiago Flores Carlos — HU-TIENDA-05 + HU-TIENDA-06 + Mejora (6 h)

Tarea	Horas
Vista /residente/tienda: grilla responsive de tarjetas (foto, nombre, precio, stock disponible). 1 columna en móvil (≤ 640 px), 2 en tablet, 3-4 en desktop. Paginación lazy (24 en 24).	2.5 h
Filtros por categoría y disponibilidad (en stock / agotado / todas) + búsqueda por nombre con debounce (300 ms). Combinables + botón 'Limpiar'.	2 h
Detalle del producto (al hacer clic): foto grande, descripción, precio, stock. Botón 'Agregar al carrito' deshabilitado con tooltip 'Disponible en la próxima versión' (placeholder del S11).	0.5 h
Indicador de stock bajo: badge naranja 'Pocas unidades' (stock ≤ 5) y badge gris 'Agotado' (stock 0) en la tarjeta.	1 h

Cortez Zamora Leonardo — Chore técnico: CD a staging (2 h)

Tarea	Horas
Workflow deploy-staging.yml: en merge a main, build (npm run build) + deploy a Vercel con VERCEL_TOKEN/ORG_ID/PROJECT_ID. El job de CD declara needs: [ci] y solo corre si lint + tests pasan.	1.5 h
Verificación post-deploy: tras el deploy, GET a /health (del S9); el workflow falla si no responde ok. Documentación en /docs/ops/cd-staging.md. Cierra el último criterio de DoD v2.	0.5 h

Riesgos del Sprint 10

#	Riesgo	Prob.	Impacto	Mitigación
R1	Subir foto de producto sin validar tipo/peso satura Storage o rompe la grilla.	● Media	● Medio	Validación cliente + servidor (JPEG/PNG, ≤ 2 MB); sin foto → placeholder. Reusa el patrón probado de solicitudes-fotos (S3).
R2	Dar de baja (DELETE) un producto con pedidos asociados rompería el historial del S11.	● Media	● Alto	La baja es lógica (activo=false), nunca DELETE. FK pedido_items → productos con ON DELETE RESTRICT. Test: no se puede borrar un producto referenciado.
R3	El deploy automático a staging publica una rama rota si CI no es gate.	● Media	● Alto	El job de CD declara needs: [ci] y solo despliega si lint + tests pasan. Verificación post-deploy con GET /health; si falla, el run se marca en rojo.
R4	Secrets de Vercel (TOKEN/ORG/PROJECT) expuestos en logs o en el repo.	● Baja	● Alto	Se cargan como GitHub Secrets, nunca en el código; el workflow los referencia con la sintaxis secrets.* y no los imprime. Revisión de PR obligatoria del workflow.

#	Riesgo	Prob.	Impacto	Mitigación
R5	El stock del catálogo del residente queda desactualizado respecto al admin.	● Baja	● Bajo	El catálogo lee stock en vivo de productos (sin caché agresiva). El descuento atómico de stock al comprar es del S11; en v1 el stock solo lo edita el admin.
R6	El residente espera comprar en el S10 (ve el botón 'Agregar al carrito').	● Media	● Bajo	El botón aparece deshabilitado con tooltip 'Disponible en la próxima versión'. Se comunica en la Review que el carrito es el S11. Gestiona expectativa.
R7	La búsqueda por nombre sin debounce dispara una consulta por cada tecla.	● Baja	● Bajo	Búsqueda con debounce (300 ms) + índice (categoria, activo). Test de que escribir rápido no satura la consulta.

2 REGISTRO DE DAILY SCRUMS

Referencia Scrum: la Daily Scrum inspecciona el progreso hacia el Sprint Goal y adapta el Sprint Backlog. Duración máxima: 15 minutos.

"Sprint Goal: Tienda v1: el admin gestiona el catálogo (alta/baja/stock/precio + foto) y el residente navega la grilla con filtros y búsqueda. Chore: CD automático a staging en cada merge a main."

Daily Scrum — Día 1 (Lunes)

DÍA	Lunes — Día 1
Progreso hacia el objetivo	Migración 011 lista: productos, pedidos y pedido_items con enums, constraints (precio/stock/cantidad, FK con ON DELETE RESTRICT) e índices. Políticas RLS verificadas con tests de los 3 roles (residente y técnico ven el catálogo activo, admin gestiona, pedidos por residente). Bucket productos-fotos creado. El spike de subida de foto (Acción 2 del Retro S9) ya dejó la subida a Storage con preview funcionando.
Plan siguiente 24h	Gonza: /admin/tienda con CRUD + subida de foto + baja lógica. Santiago: grilla /residente/tienda con tarjetas. Cortez: workflow deploy-staging.yml (build + deploy a Vercel).
Impedimentos	Discutimos si la baja de producto es DELETE o lógica. Decisión: baja lógica (activo=false) para no romper el historial de pedidos del S11; FK con ON DELETE RESTRICT. Capturado como edge case en el Refinement.
Ajuste Sprint Backlog	Sin cambios.

Daily Scrum — Día 2 (Martes)

DÍA	Martes — Día 2
Progreso hacia el objetivo	/admin/tienda operativo: Carlos da de alta 5 productos (agua, gaseosa, snacks, papel higiénico, focos LED) con foto, stock y precio; edita y da de baja. Grilla /residente/tienda mostrando las tarjetas con foto, nombre, precio y stock. Workflow de CD desplegando a Vercel en cada merge a main (probado con un merge de prueba).
Plan siguiente 24h	Gonza: pulir validaciones del CRUD (precio/stock) + placeholder sin foto. Santiago: filtros (categoría, disponibilidad) + búsqueda con debounce + indicador de stock bajo/agotado. Cortez: verificación post-deploy con GET /health + doc cd-staging.md.
Impedimentos	El deploy automático publicó una build antes de que terminara el CI. Se reordenó: el job de CD declara needs: [ci] y solo corre si pasa. Corregido en el día.
Ajuste Sprint Backlog	Sin cambios. Se usan ~30 min del buffer para el badge 'Agotado' (stock 0), sugerido por Laura en el Planning.

Daily Scrum — Día 3 (Miércoles)

DÍA	Miércoles — Día 3
Progreso hacia el objetivo	Flujo completo demostrable: Carlos gestiona el catálogo desde /admin/tienda (alta con foto, edición de precio/stock, baja lógica). Laura navega /residente/tienda: filtra por 'bebidas', busca 'agua', ve el badge 'Pocas unidades' en un producto con stock 3 y 'Agotado' en otro con stock 0. El merge a main desplegó solo a staging y la verificación post-deploy (GET /health) quedó en verde — DoD v2 cerrada del todo. El script seed:tiempo (Acción 1 del Retro S9) quedó disponible para el S11.
Plan siguiente 24h	Gonza: guión de la Sprint Review (gestionar catálogo). Santiago: validar la grilla en móvil (360 px). Cortez: ensayar el CD en pantalla compartida (merge → deploy → /health verde).
Impedimentos	Laura preguntó si podrá agregar productos al carrito; se le confirma que el carrito y la compra son el S11 (el botón ya está visible pero deshabilitado). Sin impedimento técnico.
Ajuste Sprint Backlog	Sin cambios.

3 ACTA DE SPRINT REVIEW

Referencia Scrum: la Sprint Review inspecciona el resultado del Sprint y determina adaptaciones futuras. Es una sesión de trabajo, no una presentación.

Sprint	Sprint 10 — Semana 12
Fecha	Viernes — cierre de semana 12
Duración	55 minutos
Facilitador	Alvarez Rocca Jaqueline (PO)
Scrum Team	Alvarez Rocca Jaqueline (PO), Meza Pelaez Carlos (SM), Cortez Zamora Leonardo, Gonza Morales Yoel, Santiago Flores Carlos
Stakeholders	Laura Vega (Residente — navegará y comprará en la tienda), Carlos Fuentes (Admin — gestiona el catálogo), Sra. Rosa Díaz (Dueña — observadora; le interesan los ingresos de la tienda para el dashboard ejecutivo del S14), Profesor del curso
Incremento presentado	Tienda interna v1: tablas productos/pedidos/pedido_items + RLS, /admin/tienda con CRUD del catálogo y foto a Storage, /residente/tienda con grilla, filtros y búsqueda, indicador de stock bajo/agotado, y CD a staging (deploy automático en merge a main) que cierra DoD v2. Adicionalmente, el addendum recomendado por el profesor e incorporado en el Planning (§1): seed de 3 años de datos, pago en línea simulado de facturas (HU-FACT-09) y encuesta de usabilidad (Google Form).
Modalidad de demo	Dos navegadores (admin + residente). Un merge de prueba a main en vivo para mostrar el deploy automático a staging y el GET /health en verde.

Guión de demostración

1. Carlos (admin) abre /admin/tienda y da de alta un producto: 'Agua sin gas 625 ml', categoría 'bebidas', precio \$1.50, stock 24, con foto. Toast de éxito; aparece en la grilla del catálogo.
2. Carlos edita el stock de 'Focos LED' a 3 y da de baja (activo=false) un producto discontinuado: desaparece del catálogo del residente pero sigue en el panel admin con badge 'Inactivo' y opción de reactivar.
3. Laura (residente) abre /residente/tienda y ve el catálogo en grilla de tarjetas (foto, nombre, precio, stock). El producto que Carlos acaba de crear ya aparece.
4. Laura filtra por categoría 'bebidas' y luego busca 'agua': la grilla se reduce al producto correspondiente. Pulsa 'Limpiar' y vuelve el catálogo completo.
5. Laura ve el badge naranja 'Pocas unidades' en 'Focos LED' (stock 3) y 'Agotado' (gris) en un producto sin stock. Abre el detalle: foto grande, descripción, precio. El botón 'Agregar al carrito' está deshabilitado con tooltip 'Disponible en la próxima versión'.
6. Cortez hace un merge de prueba a main en vivo: GitHub Actions corre el CI (lint + tests) y, al pasar, dispara el deploy a Vercel. Al terminar, el workflow hace GET /health y queda en verde; el cambio aparece en staging en ~1-2 min.
7. Se muestra el run de deploy-staging.yml en verde (build → deploy → verificación /health). Con esto, la verificación post-deploy automática de DoD v2 queda cerrada.

8. Addendum (mejoras del profesor): Laura paga su factura de agua desde /residente/facturas con el pago en línea simulado y pasa a 'pagada' (llega la notificación). Se muestra el dashboard con 3 años de datos de demostración ya cargados (npm run seed:historico) y el Google Form de usabilidad (SUS) abierto para recoger respuestas. Detalle en el Sprint Planning (§1).

"✓ CUMPLIDO: Tienda interna v1 operativa — /admin/tienda con CRUD del catálogo (alta/baja/stock/precio + foto), /residente/tienda con grilla, filtros y búsqueda, e indicador de stock bajo/agotado. El CD deja el deploy a staging automático en cada merge a main y cierra la verificación post-deploy de DoD v2. Sprint cerrado usando ~30 min del buffer."

Feedback de stakeholders

Laura Vega (Residente)

- «Navegar el catálogo en grilla con foto y precio se siente como una tienda de verdad; los filtros y la búsqueda son rápidos.» → Validación.
- «Me quedé con ganas de comprar — ver el botón 'Agregar al carrito' deshabilitado me dice que viene pronto.» → Expectativa gestionada (carrito = S11).
- «¿Podría ver la foto más grande al pasar el mouse, sin entrar al detalle?» → PBI-S10-E01: 'Vista rápida (quick view) del producto'. P3, 1.5 h. Candidato para Sprint 13.

Carlos Fuentes (Administrador)

- «Dar de alta un producto con foto en segundos es justo lo que necesito; la baja lógica me deja recuperar productos de temporada.» → Validación.
- «Me gustaría editar el stock de varios productos a la vez cuando llega mercadería.» → PBI-S10-E02: 'Edición de stock en lote en /admin/tienda'. P3, 2 h. Sin asignar.

Sra. Rosa Díaz (Dueña — observadora)

- «Saber que cada pedido de la tienda se sumará a la factura del mes (S11) y que todo irá al dashboard ejecutivo (S14) me deja ver el cierre del producto.» → Validación + semilla del S14.

Profesor del curso

- «Cargar 3 años de datos de demostración fue un acierto: las métricas y el futuro dashboard ejecutivo ya se ven realistas.» → Validación de MP-01 (planificada en §1).
- «Que el residente pueda pagar su factura en línea, aunque el pago sea simulado, acerca el sistema a un caso real y es un buen punto de partida para una pasarela como Culqi o Izipay.» → Validación de MP-02 / HU-FACT-09.
- «La encuesta de usabilidad (Google Form, escala Likert tipo SUS) les dará evidencia objetiva de qué tan intuitivo es el sistema.» → Validación de MP-03.

Decisiones de adaptación del Product Backlog

Decisión	Detalle
DoD v2 totalmente cerrada	El CD a staging con verificación post-deploy cierra el último criterio de DoD v2. A partir del S11 entra DoD v3.

Decisión	Detalle
Cero hotfixes	Sexto Sprint consecutivo sin hotfixes — práctica consolidada.
Epic TIENDA-01 v1 entregado	Catálogo operativo. El carrito, el descuento atómico de stock y la integración con la factura mensual son el S11 (Tienda v2).
PBI-S10-E01 (NUEVA)	Vista rápida (quick view) del producto (feedback Laura). P3, 1.5 h. Candidato S13.
PBI-S10-E02 (NUEVA)	Edición de stock en lote en /admin/tienda (feedback Carlos). P3, 2 h. Sin asignar.
CD operativo	El deploy automático a staging queda activo. Los smoke tests post-deploy más completos entran como chore del S14.
Mejoras del profesor entregadas	Las 3 mejoras recomendadas por el profesor e incorporadas en el Planning (§1) — MP-01 seed de 3 años, MP-02 pago en línea simulado de facturas (HU-FACT-09) y MP-03 encuesta de usabilidad (Likert/SUS) — se entregaron dentro del Sprint 10 como addendum (≈8 h), sin alterar el compromiso base de 13 h de la Tienda v1.
Sprint 11 confirmado	Tienda v2 (DoD v3): carrito del residente + descuento atómico de stock al confirmar + el pedido se suma como línea a la factura mensual + vista admin de pedidos. Chore: privacidad / no-PII en logs de tienda y factura.

4 ACTA DE SPRINT RETROSPECTIVE

Referencia Scrum: la Retrospective planifica formas de aumentar calidad y efectividad. Los cambios más impactantes pueden entrar al Sprint Backlog del próximo Sprint.

Sprint	Sprint 10 — Semana 12
Fecha	Viernes — después de la Sprint Review
Duración	30 minutos
Facilitador	Meza Pelaez Carlos (Scrum Master)
Participantes	Todo el Scrum Team (PO + SM + Developers)

✓ ¿Qué salió bien?

- La Acción 2 del Retro S9 (spike de librería en el Refinement) funcionó: el spike de subida de fotos a Storage del viernes hizo que el CRUD del lunes saliera sin fricción.
- Reusar el patrón de Storage del S3 (solicitudes-fotos) para productos-fotos ahorró tiempo: misma validación, mismas URLs firmadas.
- La decisión temprana de baja lógica (activo=false) en vez de DELETE evitó un problema de integridad con los pedidos del S11.
- El CD quedó con CI como gate y verificación /health post-deploy — la deuda de 'deploy automático' que arrastrábamos desde el S8 quedó saldada.
- Tener a Laura en el Planning (segunda vez) capturó el badge 'Agotado' a tiempo, dentro del buffer.
- Se incorporaron las tres mejoras recomendadas por el profesor (seed de 3 años, pago simulado de facturas y encuesta de usabilidad) como addendum, sin afectar el compromiso ni la velocidad del núcleo del Sprint.
- Sexto Sprint consecutivo sin hotfixes.

✗ ¿Qué salió mal?

- El primer intento de CD desplegó antes de que terminara el CI (faltó el needs: entre jobs). Se detectó el Día 2 y se corrigió, pero un workflow de CD debió revisarse con más cuidado antes de mergear a main.
- El indicador de stock bajo / 'Agotado' generó debate sobre el umbral (≤ 5 vs ≤ 3). Se fijó ≤ 5 y se documentó en conventions.md, pero pudo capturarse en el Refinement.
- La grilla en móvil necesitó un ajuste de columnas de último momento; faltó validar el breakpoint en el spike.

Acciones de mejora (máx. 3)

Acción 1 — Checklist de revisión para workflows de CI/CD

Descripción	Antes de mergear un workflow nuevo de GitHub Actions, pasar una checklist (gate de CI, needs: entre jobs, secrets referenciados, verificación post-deploy). Evita publicar builds rotas a staging.
Dueño	Cortez Zamora Leonardo

Evidencia	/docs/ops/checklist-workflows.md aplicada al siguiente workflow (smoke tests del S14)
Fecha	Sprint 11

Acción 2 — Umbrales y constantes de UI en conventions.md

Descripción	Documentar en /docs/conventions.md los umbrales y constantes de presentación (stock bajo ≤ 5 , debounce 300 ms, peso máx. de foto 2 MB) para no re-discutirlos. Revisar en el Refinement.
Dueño	Santiago Flores Carlos
Evidencia	conventions.md con la sección 'Constantes de UI' (≥ 3 entradas)
Fecha	Sprint 11

Acción 3 — Validar breakpoints móviles en el spike, no al final

Descripción	Cuando un PBI tiene grilla o layout responsive, el spike del Refinement valida los breakpoints (360 / 768 / 1024 px) antes del Sprint, no en el último Daily.
Dueño	Meza Pelaez Carlos
Evidencia	El acta del Refinement del S11 (carrito) incluye validación de breakpoints
Fecha	Sprint 11

Verificación DoD v2

Criterio	Estado
Todo lo de DoD v1 (lint, unit tests, README, manejo errores, seed, deploy preview)	✓ CUMPLIDO
Pruebas de integración para flujos críticos (RLS de tienda con 3 roles + CRUD de productos)	✓ CUMPLIDO — 8 tests de integración nuevos
Cobertura $\geq 60\%$ en módulos core (src/lib/tienda.ts)	✓ CUMPLIDO — 71%
Endpoint /health disponible en staging	✓ CUMPLIDO — usado por el CD como verificación post-deploy
Variables de entorno vía Secrets CI/CD — sin hardcode	✓ CUMPLIDO — VERCEL_TOKEN/O RG_ID/PROJECT_ID en GitHub Secrets
Despliegue staging con verificación post-deploy	✓ CUMPLIDO — CD automático en merge a main + GET /health (cierra el último criterio de DoD v2)
tsc --noEmit pasa sin errores en CI	✓ CUMPLIDO

Criterio	Estado
ADR-011 (modelo de Tienda) + ADR-012 (CD a staging) documentados	✓ CUMPLIDO — adicional a DoD v2

"DoD v2 cerrada al 100% en sus criterios core: el CD con verificación post-deploy cierra el único matiz que quedaba desde el S8. A partir del Sprint 11 (Tienda v2) entra DoD v3, que exige no-PII en logs y entrega como release candidate. Sprint 10 cerrado usando ~30 min del buffer; sexto Sprint consecutivo sin hotfixes."

5 HISTORIAS DE USUARIO — SPRINT 10

Módulo TIENDA INTERNA — Sprint 10 (v1)

HU-TIENDA-01 · Modelado BD de la Tienda · Tablas + RLS + bucket

Sprint 10 · 2 h

P1 · 2 h

Como **sistema**, quiero **las tablas productos, pedidos y pedido_items con RLS por rol y un bucket de fotos**, para soportar el catálogo (v1) y, más adelante, el carrito y la integración con la factura (v2, S11).

Criterios de aceptación

- Migración 011 crea productos (id, nombre, descripcion, categoria enum, precio numeric(10,2) ≥ 0 , stock int ≥ 0 , activo boolean, imagen_url, timestamps), pedidos (id, residente_id, estado enum borrador/confirmado/facturado, total, periodo) y pedido_items (id, pedido_id, producto_id, cantidad ≥ 1 , precio_unitario).
- Constraints: precio ≥ 0 , stock ≥ 0 , cantidad ≥ 1 . FK pedido_items $\rightarrow$ productos con ON DELETE RESTRICT (no se borra un producto referenciado).
- Índices por (categoria, activo) para el catálogo y (residente_id) para pedidos.
- RLS: productos SELECT para autenticados donde activo=true; admin SELECT/INSERT/UPDATE en todo. pedidos/pedido_items: residente los suyos, admin todos, técnico sin acceso.
- Bucket productos-fotos en Storage con políticas (admin escribe, lectura autenticada) — reusa el patrón de solicitudes-fotos (S3).
- Tests de integración con los 3 roles validan las políticas RLS.

Evidencia: Migración 011 aplicada en staging; RLS verificada con tests automatizados; ADR-011 documenta el modelo.

HU-TIENDA-02 · Admin gestiona el catálogo · CRUD + foto + baja lógica

Sprint 10 · 3 h

P1 · 3 h

Como **administrador**, quiero **dar de alta, editar y dar de baja productos con foto, stock y precio**, para mantener el catálogo de la tienda del edificio.

Criterios de aceptación

- Vista /admin/tienda con listado de productos (activos e inactivos) + formulario de alta/edición (nombre, descripción, categoría, precio, stock, foto).
- Subida de foto a productos-fotos (JPEG/PNG, ≤ 2 MB) con preview; sin foto $\rightarrow$ placeholder.
- Baja lógica: 'Dar de baja' marca activo=false (no DELETE); el producto desaparece del catálogo del residente pero queda en el panel admin con badge 'Inactivo' y puede reactivarse.
- Validaciones cliente y servidor: precio ≥ 0 , stock ≥ 0 , nombre obligatorio.
- Toda alta/edición/baja registra en audit_log.

Evidencia: Carlos dio de alta 5 productos con foto y dio de baja uno en la Review; el inactivo desapareció del catálogo del residente y quedó reactivable en el panel admin.

Como **residente**, quiero **navegar el catálogo de la tienda en una grilla de tarjetas con foto, nombre, precio y stock**, para ver qué puedo pedir.

Criterios de aceptación

- Vista /residente/tienda con grilla de tarjetas: foto, nombre, precio y stock disponible. Solo productos con activo=true.
- Responsiva: 1 columna en móvil (≤ 640 px), 2 en tablet, 3-4 en desktop.
- Detalle del producto al hacer clic: foto grande, descripción, precio, stock. Botón 'Agregar al carrito' deshabilitado con tooltip 'Disponible en la próxima versión' (placeholder del S11).
- Paginación lazy (carga de 24 en 24) para soportar catálogos grandes.
- RLS verifica que solo se muestren productos con activo=true.

Evidencia: Laura navegó el catálogo en grilla durante la Review; el producto recién creado por Carlos apareció al instante.

Como **residente**, quiero **filtrar el catálogo por categoría y disponibilidad y buscar por nombre**, para encontrar rápido lo que necesito.

Criterios de aceptación

- Filtro por categoría (bebidas / comestibles / limpieza / otros / todas) en la cabecera del catálogo.
- Filtro por disponibilidad (en stock / agotado / todas).
- Búsqueda por nombre con debounce (300 ms); filtra sobre el catálogo activo.
- Los filtros y la búsqueda se combinan; el botón 'Limpiar' restablece la grilla completa.
- La consulta usa el índice (categoría, activo); escribir rápido no dispara una consulta por cada tecla.

Evidencia: Laura filtró por 'bebidas' y buscó 'agua' en la Review; la grilla se redujo al producto correspondiente.

Como **residente**, quiero **ver de un vistazo qué productos tienen pocas unidades o están agotados**, para no llevarme una sorpresa al pedir.

Criterios de aceptación

- Badge naranja 'Pocas unidades' en la tarjeta cuando stock ≤ 5 (umbral en conventions.md).
- Badge gris 'Agotado' cuando stock = 0; la tarjeta se atenúa.
- El badge es puramente visual; no bloquea (la validación de compra es del S11). Stock 0 sí impedirá agregar al carrito en el S11.

Evidencia: En la Review, 'Focos LED' (stock 3) mostró 'Pocas unidades' y un producto sin stock mostró 'Agotado'.

Chore técnico del Sprint: CD a staging con GitHub Actions (deploy-staging.yml): en cada merge a main, tras pasar el CI (lint + tests), se hace build y deploy a Vercel con VERCEL_TOKEN/ORG_ID/PROJECT_ID y una verificación post-deploy con GET /health (del S9). Cierra el último criterio de DoD v2. Sin smoke tests más completos por ahora (entran como chore del S14). 2 h, P2. ADR-012 documenta la decisión.

PBIs emergentes — entran en Sprints futuros

ID	Historia	Prior.	Horas est.	Sprint
PBI-S10-E01	Vista rápida (quick view) del producto (feedback Laura)	● P3	1.5 h	13
PBI-S10-E02	Edición de stock en lote en /admin/tienda (feedback Carlos)	● P3	2 h	Sin asignar
PBI-S9-E01	Exportar totales de facturación a CSV	● P3	1.5 h	13
PBI-S9-E02	Recordatorio también el día del vencimiento	● P2	1 h	12
PBI-S6-E03	Sesiones activas + cerrar todas al cambiar contraseña	● P2	2 h	13
PBI-S6-E01	Indicador de presencia en panel admin (Supabase Presence)	● P3	2 h	12
PBI-S6-E02	Sonido + haptic feedback opcional al recibir notificación	● P3	2 h	12
PBI-S7-E01	Filtros por categoría en el dashboard de métricas	● P2	1 h	13

6 ESTADO DEL BACKLOG TRAS SPRINT 10

Progreso acumulado

Sprint	Horas	Incremento entregado	Estado
Sprint 0	12 h	Setup técnico + CI + ADRs + Supabase configurado	✓ Completado
Sprint 1	15 h	Módulo Auth: registro 2 pasos + login por rol + recuperación	✓ Completado
Sprint 2	14 h	Panel admin + gestión de usuarios + bloqueo/desbloqueo	✓ Completado
Sprint 3	13 h	Mantenimiento v1: crear solicitud con foto + vista admin	✓ Completado
Sprint 4	13 h	Mantenimiento v2: asignación + vista técnico + cierre + cámara + confirmación	✓ Completado
Sprint 5	13 h	Trazabilidad: audit log + historial + perfil + foto al rechazar	✓ Completado
Sprint 6	13 h	Comunicación Realtime: notificaciones + email + foto cierre + alerta admin	✓ Completado
Sprint 7	13 h	Métricas y Dashboard visual: KPIs + gráficas Recharts + CSV + vista materializada	✓ Completado
Sprint 8	13 h	Facturación v1: tabla facturas + emisión individual/lote + vista residente + notificación Realtime	✓ Completado
Sprint 9	12.5 h	Facturación v2: marcar pagada + recordatorios + vencida + comprobante PDF + totales + /health	✓ Completado
Sprint 10	13 + 8 h	★ Tienda interna v1: catálogo admin (CRUD + foto) + grilla residente con filtros/búsqueda + CD a staging · Addendum del profesor: seed 3 años + pago simulado de facturas + encuesta de usabilidad	✓ Completado
Sprint 11	13 h (est.)	Tienda interna v2: carrito + descuento atómico + pedido → factura mensual + vista admin de pedidos (primer DoD v3)	→ Próximo
Sprints 12–14	39 h (est.)	Notificaciones avanzadas · Panel integral residente · ★ Dashboard ejecutivo + RC	■ Planificado

Total invertido: 144.5 horas en 11 sprints (Sprint 0 → Sprint 10), más 8 h del addendum de mejoras del profesor incorporado al Sprint 10 (152.5 h en total).

Total restante: ~52 horas estimadas en 4 sprints.

Velocidad promedio: 13.1 horas/sprint (estable por octavo sprint consecutivo).

Roadmap actualizado tras Sprint 10 Review

Sprint	Sem.	Objetivo principal — Entregable funcional
S11	13	Tienda interna v2: carrito + descuento atómico de stock + pedido se suma a la factura mensual + vista admin de pedidos (primera aplicación de DoD v3)
S12	14	Notificaciones avanzadas: Web Push + presencia + sonido/haptic + preferencias (absorbe PBI-S9-E02: recordatorio el día del vencimiento)
S13	15	Panel integral del residente + sesiones activas + filtros por categoría en métricas (S7-E01) + export CSV de totales (S9-E01) + quick view de producto (S10-E01)
S14	16	★ Dashboard ejecutivo del dueño (Mantenimiento + Finanzas + Tienda) + Release Candidate + demo final integral

Nota: los emergentes del S10 se ubican — PBI-S10-E01 (quick view) en S13 y PBI-S10-E02 (edición de stock en lote) sin asignar. La Tienda v1 deja el catálogo listo; el S11 cierra el ciclo con el carrito y la integración con la factura mensual del residente (reusa la estructura de facturas del S9).

Vista previa Sprint 11 — Tienda interna v2 (carrito + integración con factura)

"Sprint 11: cerrar el ciclo de la tienda. El residente arma un carrito y confirma su pedido; el pedido descuenta stock de forma atómica y, al cierre de mes, se suma como línea a su factura. El admin ve todas las órdenes con filtros. Primera aplicación de DoD v3 (no-PII en logs + release candidate)."

PBI	Historia	Horas est.	Prior.
HU-TIENDA-03	Carrito del residente + confirmación de pedido (descuento atómico de stock)	3 h	● P1
HU-TIENDA-04	El pedido genera línea automática en la factura mensual del residente	3 h	● P1
HU-TIENDA-07	Historial de pedidos del residente en /residente/tienda/historial	2 h	● P2
HU-TIENDA-08	Vista admin de pedidos (todas las órdenes, filtros por residente/fecha/estado)	2 h	● P2
Mejora	Mini-carrito en navbar con badge de unidades agregadas y subtotal	1 h	● P3
Chore-T	Privacidad / no-PII: revisión de logs de tienda y factura (solo IDs)	2 h	● P2

Total estimado Sprint 11: 13 horas (2 h de buffer).

Variables de entorno acumuladas al Sprint 10

Este Sprint añade 3 variables para el CD (Secrets de GitHub/Vercel). Las anteriores siguen vigentes:

Variable	Descripción	Dónde se usa	Desde
VITE_SUPABASE_URL	URL pública del proyecto Supabase	Frontend (cliente JS) · dummy en CI	Sprint 0

Variable	Descripción	Dónde se usa	Desde
VITE_SUPABASE_ANON_KEY	Clave anónima de Supabase	Frontend (cliente JS) · dummy en CI	Sprint 0
SUPABASE_SERVICE_ROLE_KEY	Clave de servicio (admin) de Supabase	Edge Functions (servidor) · /health	Sprint 3
RESEND_API_KEY	Clave de API de Resend para emails	Edge Functions (recordatorio / factura)	Sprint 2
RESEND_FROM_ADDRESS	Remitente de los emails de notificaciones	Edge Function recordatorios-facturas	Sprint 6
SUPABASE_DB_URL	URL de conexión directa a la BD	Migraciones locales · pg_cron	Sprint 0
VERCEL_TOKEN	Token de despliegue de Vercel	GitHub Actions (CD a staging)	Sprint 10 (nuevo)
VERCEL_ORG_ID	ID de la organización en Vercel	GitHub Actions (CD a staging)	Sprint 10 (nuevo)
VERCEL_PROJECT_ID	ID del proyecto en Vercel	GitHub Actions (CD a staging)	Sprint 10 (nuevo)

Variables proyectadas para futuros Sprints:

- Sprint 12 (Web Push): VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY para las notificaciones push.

Ninguna de estas variables debe aparecer en el código fuente ni en commits. Todas van en .env.local (local) y en Secrets de GitHub/Vercel (CI/CD).

— Zity · Artefactos Sprint 10 · Documento vivo — actualizar en cada Sprint Review —